Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes
Département Ingénierie des Systèmes Embarqués
Filière Smart Systems Engineering

TP 1 : Comprendre la station – Mise en service et caractérisation des trois nœuds d'EcoTri 4.0

Projet fédérateur EcoTri 4.0 – Sous-projet SP1 – Jalon J1

Pr.Hassan BERBIA/Pr.Aissam BERRAHOU

31 octobre 2026

Objectifs du TP

Ce TP prolonge le Tutorial 1. Chaque équipe (3 ou 4 étudiants) met en service les trois cartes de la station et **mesure** leurs performances de capture, afin de **justifier par des chiffres** le placement des traitements dans l'architecture.
À la fin de ce TP, vous serez capable de :

- Programmer une capture d'image sur la Portenta H7 (OpenMV), la XIAO ESP32-S3 Sense et l'ESP32-CAM (Arduino).
- Mesurer FPS, temps de capture, taille d'image et mémoire libre selon la résolution et le format.
- Confronter les mesures aux formules du cours ($M = WHb$, $D = WHbf$, $T_{\max} = d/v$).
- Rédiger un cahier des charges chiffré et un schéma d'architecture justifié.

Livrable – Jalon J1 (à déposer en fin de séance) : cahier des charges, schéma d'architecture et tableau des mesures (partie 5).

1 Contexte : la station EcoTri 4.0

La station reçoit des déchets sur un tapis à $v = 0{,}5$ m/s ; l'éjecteur est à $d = 30$ cm de la caméra. Trois cartes se partagent le travail :

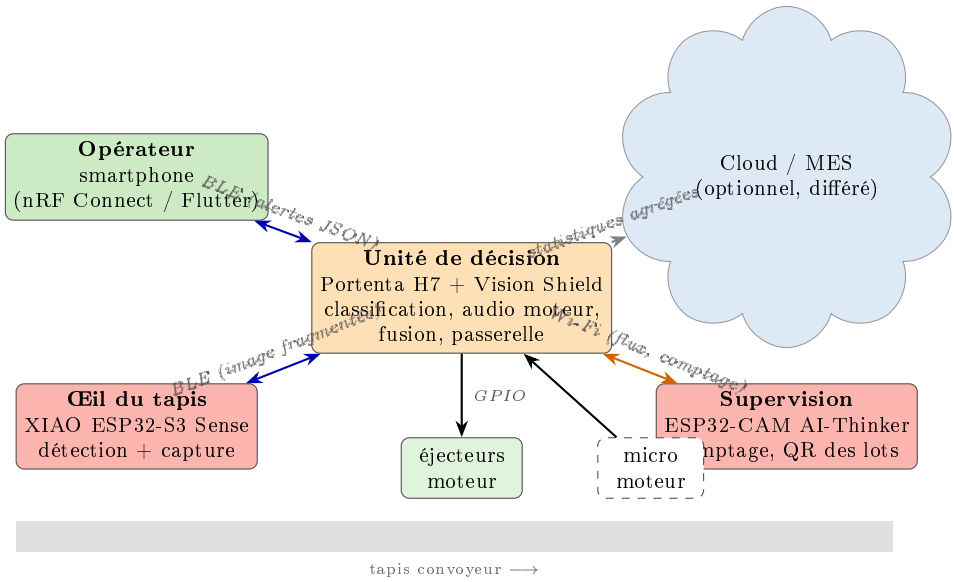


Figure 1: *
Architecture cible. Ce TP caractérise les trois nœuds ; les liens BLE et Wi-Fi seront réalisés à la séance 3.

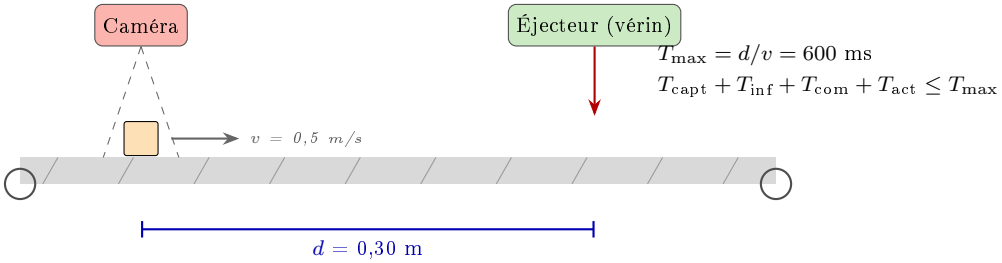


Figure 2: *
Contrainte temporelle : la décision doit être prise en moins de $T_{\max} = 600$ ms.

2 Pré-requis

Matériel (par équipe)

- **Arduino Portenta H7** + **Vision Shield**, câble USB-C.
- **Seeed XIAO ESP32-S3 Sense** (carte d'extension caméra montée), câble USB-C.
- **ESP32-CAM AI-Thinker** + adaptateur USB-série (ESP32-CAM-MB ou FTDI 3,3 V).
- Quelques objets du fil rouge : canette, bouteille plastique, carton, bocal en verre.

Logiciel

1. **OpenMV IDE** (4.0 ou plus) – Tutorial 1 terminé.
2. **Arduino IDE 2.x** avec le paquet de cartes **esp32 by Espressif Systems** (version 2.0.x ou 3.x) :

- URL du gestionnaire de cartes : https://espressif.github.io/arduino-esp32/package_esp32_index.j
- XIAO : carte **XIAO_ESP32S3**, option **Tools** → **PSRAM : OPI PSRAM**.
- ESP32-CAM : carte **AI Thinker ESP32-CAM**.

3 Étapes détaillées du TP

Partie 1 : Portenta H7 – banc de mesure OpenMV (30 min)

Le script suivant automatise les mesures du Tutorial 1 : pour chaque résolution, il capture N images, chronomètre la capture seule, puis une capture suivie d'un traitement léger (histogramme), et affiche une ligne CSV.

```
# TP1 - banc de mesure Portenta H7 + Vision Shield (OpenMV)
import sensor, time, gc
from pyb import LED

N = 60
TAILLES = [("96x96",    sensor.QVGA,  (112, 72, 96, 96)),   # fenêtrage
           ("QQVGA",    sensor.QQVGA, None),
           ("QVGA",     sensor.QVGA,  None),
           ("320x320",  sensor.B320X320, None)]

def mesure(nom, taille, fenetre):
    sensor.set_framesize(taille)
    if fenetre: sensor.set_windowing(fenetre)
    sensor.skip_frames(time=800)
    # 1) capture seule
    t0 = time.ticks_us()
    for _ in range(N):
        img = sensor.snapshot()
    t_capt = time.ticks_diff(time.ticks_us(), t0) / N / 1000
    # 2) capture + traitement léger (statistiques de l'image)
    t0 = time.ticks_us()
    for _ in range(N):
        img = sensor.snapshot()
        moy = img.get_statistics().mean()
    t_tot = time.ticks_diff(time.ticks_us(), t0) / N / 1000
    gc.collect()
    print("%s;%d;%d;%d;%.2f;%.1f;%.2f;%d" % (nom, img.width(), img.height(), img.size
    (),
          t_capt, 1000 / t_capt, t_tot, gc.mem_free()))

sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)
LED(3).on()
print("resolution;W;H;octets;capture_ms;fps;capture+stats_ms;tas_libre")
for nom, taille, fenetre in TAILLES:
    try:
        mesure(nom, taille, fenetre)
    except Exception as e:
        print(nom, ";erreur;", e)
    sensor.reset(); sensor.set_pixformat(sensor.GRAYSCALE)
LED(3).off(); LED(2).on()
```

1. Exécutez le script **avec le Frame Buffer désactivé**. Copiez la sortie CSV dans votre tableur.
2. Refaites la mesure QVGA avec l'affichage activé. Quel est le surcoût de l'IDE ?

3. Ajoutez `img.save("/test.jpg", quality=85)` dans la boucle QVGA : quel est le coût d'écriture en Flash ?

Partie 2 : XIAO ESP32-S3 Sense et ESP32-CAM – capture Arduino (45 min)

Un seul sketch sert pour les deux cartes ; seule la table des broches change (décommentez la bonne ligne `#define`). Il parcourt plusieurs résolutions et formats et affiche les mesures sur le moniteur série (115200 bauds).

```
// TP1 - banc de mesure caméra ESP32 (XIAO ESP32-S3 Sense / ESP32-CAM AI-Thinker)
#include "esp_camera.h"

#define CARTE_XIAO_S3
// #define CARTE_ESP32CAM

#if defined(CARTE_XIAO_S3)
  #define PWDN -1
  #define RESET -1
  #define XCLK 10
  #define SIOD 40
  #define SIOC 39
  #define Y9 48
  #define Y8 11
  #define Y7 12
  #define Y6 14
  #define Y5 16
  #define Y4 18
  #define Y3 17
  #define Y2 15
  #define VSYNC 38
  #define HREF 47
  #define PCLK 13
#elif defined(CARTE_ESP32CAM)
  #define PWDN 32
  #define RESET -1
  #define XCLK 0
  #define SIOD 26
  #define SIOC 27
  #define Y9 35
  #define Y8 34
  #define Y7 39
  #define Y6 36
  #define Y5 21
  #define Y4 19
  #define Y3 18
  #define Y2 5
  #define VSYNC 25
  #define HREF 23
  #define PCLK 22
#endif

const int N = 40;

bool initCamera(framesize_t taille, pixformat_t format) {
  esp_camera_deinit();
  camera_config_t c = {};
  c.pin_pwdn = PWDN;  c.pin_reset = RESET; c.pin_xclk = XCLK;
  c.pin_sccb_sda = SIOD; c.pin_sccb_scl = SIOC;
  c.pin_d7 = Y9; c.pin_d6 = Y8; c.pin_d5 = Y7; c.pin_d4 = Y6;
```

```
  c.pin_d3 = Y5; c.pin_d2 = Y4; c.pin_d1 = Y3; c.pin_d0 = Y2;
  c.pin_vsync = VSYNC; c.pin_href = HREF; c.pin_pclk = PCLK;
  c.xclk_freq_hz = 20000000;
  c.ledc_timer = LEDC_TIMER_0; c.ledc_channel = LEDC_CHANNEL_0;
  c.pixel_format = format;
  c.frame_size = taille;
  c.jpeg_quality = 12;                    // 0-63 : plus petit = meilleure qualité
  c.fb_count = 1;
  c.fb_location = psramFound() ? CAMERA_FB_IN_PSRAM : CAMERA_FB_IN_DRAM;
  c.grab_mode = CAMERA_GRAB_LATEST;
  return esp_camera_init(&c) == ESP_OK;
}

void mesurer(const char* nom, framesize_t taille, pixformat_t format) {
  size_t sram_avant = heap_caps_get_free_size(MALLOC_CAP_INTERNAL);
  size_t psram_avant = ESP.getFreePsram();
  if (!initCamera(taille, format)) { Serial.printf("%s;erreur init\n", nom); return;
   }
  for (int i = 0; i < 5; i++) { camera_fb_t* fb = esp_camera_fb_get();
   esp_camera_fb_return(fb); }
  size_t octets = 0;
  uint32_t t0 = micros();
  for (int i = 0; i < N; i++) {
    camera_fb_t* fb = esp_camera_fb_get();
    if (!fb) { Serial.println("capture échouée"); return; }
    octets += fb->len;
    esp_camera_fb_return(fb);
  }
  float t_ms = (micros() - t0) / 1000.0 / N;
  Serial.printf("%s;%u;%.2f;%.1f;%u;%u;%u;%u\n", nom, (unsigned)(octets / N), t_ms,
   1000.0 / t_ms,
                (unsigned)sram_avant, (unsigned)heap_caps_get_free_size(
   MALLOC_CAP_INTERNAL),
                (unsigned)psram_avant, (unsigned)ESP.getFreePsram());
}

void setup() {
  Serial.begin(115200);
  delay(3000);
  Serial.printf("CPU %d MHz | Flash %u o | PSRAM %u o\n",
                getCpuFrequencyMhz(), ESP.getFlashChipSize(), ESP.getPsramSize());
  Serial.println("config;octets_image;capture_ms;fps;sram_avant;sram_apres;
   psram_avant;psram_apres");
  mesurer("96x96_GRAY",  FRAMESIZE_96X96, PIXFORMAT_GRAYSCALE);
  mesurer("QQVGA_GRAY",  FRAMESIZE_QQVGA, PIXFORMAT_GRAYSCALE);
  mesurer("QVGA_GRAY",   FRAMESIZE_QVGA,  PIXFORMAT_GRAYSCALE);
  mesurer("QVGA_RGB565", FRAMESIZE_QVGA,  PIXFORMAT_RGB565);
  mesurer("QVGA_JPEG",   FRAMESIZE_QVGA,  PIXFORMAT_JPEG);
  mesurer("VGA_JPEG",    FRAMESIZE_VGA,   PIXFORMAT_JPEG);
  mesurer("VGA_RGB565",  FRAMESIZE_VGA,   PIXFORMAT_RGB565);
}

void loop() {}
```

- `esp_camera_fb_get()` renvoie un **frame buffer** dont la taille `fb->len` est mesurée ; en JPEG, elle varie selon la scène.
- `heap_caps_get_free_size(MALLOC_CAP_INTERNAL)` donne la SRAM interne libre, `ESP.getFreePsram()` la PSRAM libre : comparez avant et après l'initialisation de la caméra pour voir où le frame buffer

a été alloué.

Téléversement :

1. **XIAO** : branchez en USB-C, sélectionnez la carte **XIAO_ESP32S3**, **PSRAM : OPI PSRAM**, puis téléversez. Si le port n'apparaît pas, maintenez **BOOT** en branchant le câble.
2. **ESP32-CAM** : posez-la sur l'adaptateur ESP32-CAM-MB, sélectionnez **AI Thinker ESP32-CAM**. Si le téléversement échoue (« Failed to connect »), maintenez **IO0** pendant la connexion puis appuyez sur **RST** ; après le téléversement, appuyez sur **RST** pour lancer le programme.

Astuce

Certaines combinaisons échouent volontairement (par exemple VGA RGB565 sans PSRAM : 614 400 octets ne tiennent pas en SRAM). Notez l'erreur : c'est une **mesure** au même titre qu'un FPS, et elle illustre le cours.

Partie 3 : Analyse des mesures (20 min)

1. Remplissez le tableau de la partie 5 pour les trois cartes.
2. Comparez la taille mesurée des images brutes à la formule $M = W \times H \times b$. Expliquez tout écart.
3. Calculez le taux de compression JPEG en QVGA ($M_{\text{brut}}/M_{\text{JPEG}}$) sur une scène du tapis avec et sans objet.
4. Tracez (tableur ou Python) le FPS en fonction du nombre de pixels pour chaque carte. La relation est-elle linéaire ? Pourquoi ?
5. Pour chaque carte, calculez le débit D qu'il faudrait pour transmettre le flux QVGA JPEG à la cadence mesurée, et comparez à 1 Mbit/s (ordre de grandeur du BLE).

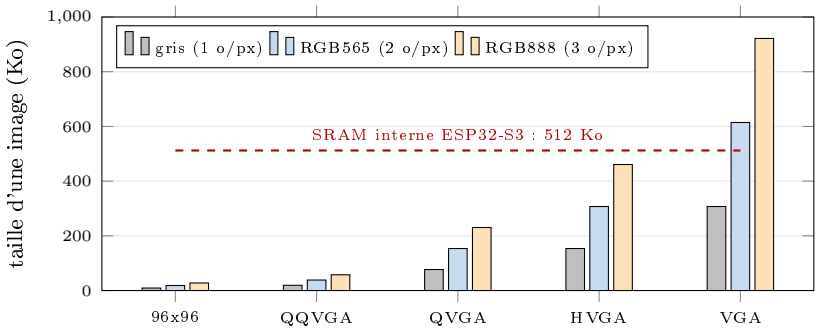


Figure 3: *
Valeurs théoriques de M : vos mesures doivent retrouver ces ordres de grandeur.

Question

Sur l'ESP32-CAM, le frame buffer QVGA RGB565 (153 600 octets) a-t-il été placé en SRAM interne ou en PSRAM ? Que se passerait-il en ajoutant un modèle dont le pic d'activations est de 250 Ko ?

Partie 4 : Budget de latence de la station (15 min)

Avec vos mesures, complétez le budget suivant pour l'unité de décision (Portenta, QVGA ou 96×96) :

Étape	Temps (ms)	Origine
Capture T_{capt}		mesuré (partie 1)
Prétraitement $T_{\text{pré}}$		mesuré (statistiques, partie 1)
Inférence T_{inf}		estimé : $N_{\text{MAC}}/(\eta f_{\text{CPU}})$ avec 6 M MAC, $\eta = 1{,}5$
Communication T_{com}	50	hypothèse (BLE, séance 3)
Actionnement T_{act}	80	hypothèse (vérin)
Total		**à comparer à 600 ms**

Question

Quelle marge reste-t-il ? Quelle vitesse de tapis maximale la station pourrait-elle supporter avec cette chaîne ? Refaites le calcul pour un scénario « tout cloud » avec un RTT 4G de 150 ms en moyenne et 700 ms au 99e centile.

Partie 5 : Livrable – Jalon J1 (20 min)

a) Tableau des mesures (une ligne par configuration testée)

Carte	Résolution / format	Octets / image	Capture (ms)	FPS	SRAM libre (o)	PSRAM / tas libre (o)
Portenta H7	96×96 gris					
Portenta H7	QVGA gris					
XIAO S3	QVGA gris					
XIAO S3	QVGA JPEG					
XIAO S3	VGA RGB565					
ESP32-CAM	QVGA gris					
ESP32-CAM	QVGA JPEG					
ESP32-CAM	VGA RGB565					

b) Cahier des charges : rédigez au moins **3 exigences fonctionnelles** (EF-1, EF-2, ...) et **4 exigences non fonctionnelles chiffrées** (ENF-1, ...) avec, pour chacune, sa **méthode de vérification**. Exemple :

Id	Exigence	Vérification
ENF-1	Latence capture→décision $\leq$ 600 ms au 99e centile	chronométrage de 500 objets
ENF-3	Aucune image ne quitte la station	revue du protocole + capture réseau
...		

c) Schéma d'architecture : reprenez le schéma de la partie 1 et, pour **chaque traitement**

(détection, capture, classification, comptage, analyse audio, statistiques), indiquez la carte qui l'exécute et justifiez ce choix avec au moins **un chiffre** issu de vos mesures ou du budget (latence, débit, mémoire, énergie).

4 Grille d'évaluation du jalon J1

Critère	**Attendu**	**Points**
Fonctionnement	les trois cartes capturent ; scripts fournis et commentés	4
Mesures	tableau complet, conditions précisées (IDE, format), écarts expliqués	6
Cahier des charges	exigences numérotées, chiffrées, vérifiables	4
Architecture	placement justifié par des chiffres (grille de décision du cours)	4
Qualité	clarté du rendu, unités, sources	2
Total		**20**

5 Ressources

- Documentation OpenMV : docs.openmv.io.
- Espressif, pilote caméra `esp32-camera` : github.com/espressif/esp32-camera (exemple *CameraWebServer*, fichier `camera_pins.h`).
- Seeed Studio, *Getting Started with XIAO ESP32S3 (Sense)* : wiki.seeedstudio.com.
- Harvard *Machine Learning Systems* (V. Janapa Reddi) – chapitre « Embedded AI » : mlsysbook.ai.